SQL для аналитика

Handbook по материалам лекции «SQL. Введение»

Школа Аналитиков · hh.ru

> **СОВЕТ**
> Это справочник, а не конспект. Используйте оглавление, чтобы быстро найти нужный синтаксис. Учебная база для всех примеров — hh_school.

Оглавление

- 1. Термины: БД, СУБД, SQL
- 2. DBeaver: горячие клавиши
- 3. Как устроены таблицы
- 4. Первый SQL: SELECT, WHERE, ORDER BY, LIMIT
- 5. WHERE, ORDER BY, LIMIT
- 6. Агрегации и метрики: GROUP BY, HAVING
- 7. JOIN: соединение таблиц
- 8. Функции для аналитика: даты, CASE, NULL, FILTER
- 9. Шпаргалка

1. Термины: БД, СУБД, SQL

Любая аналитика начинается с одного и того же вопроса — «где взять данные?». В реальной работе файла с готовой выгрузкой нет: данные лежат в базе, и достать их оттуда помогает SQL.

База данных, СУБД, SQL

- **База данных (БД)** — организованное хранилище: данные можно хранить, связывать между собой и быстро находить
- **СУБД (система управления базами данных)** — программа-посредник: принимает запросы, находит данные, следит за целостностью и правами доступа. Примеры: PostgreSQL, MySQL, ClickHouse, Oracle
- **SQL (Structured Query Language)** — язык структурированных запросов. Он декларативный: в запросе описывается, что нужно получить, а не как это искать — в этом отличие от языков вроде Python

> **СОВЕТ**
> БД — это библиотека, СУБД — библиотекарь, а SQL — язык, на котором мы с ним разговариваем.

Компоненты SQL

Компонент	Назначение
DDL — Data Definition Language	Определяет структуру БД: создаёт, изменяет, удаляет её компоненты
DML — Data Manipulation Language	Работает с данными в таблицах: добавляет, изменяет, удаляет, извлекает
DCL — Data Control Language	Безопасность: предоставляет или отзывает права на работу с БД

Типы баз данных

Тип	Как устроены данные	Пример
Реляционная	Таблицы со строками и колонками, есть связи между таблицами	Пользователи, вакансии, отклики
Документная	Отдельные документы, например JSON	Профили пользователей
Ключ–значение	Пары ключ → значение	Сессия пользователя, кэш
Графовая	Объекты и связи между ними	Соцсеть: кто с кем дружит

Для аналитика особенно важны реляционные БД — именно в них чаще всего работают с таблицами, связями и SQL. Сегодня во всех примерах используется именно она.

Способы хранения

Способ	Как хранятся данные	Примеры
Строковое	Данные одной записи хранятся рядом — удобно для отдельных записей	PostgreSQL, MySQL
Колоночное	Значения одного столбца хранятся рядом — удобно для больших объёмов	ClickHouse, DuckDB

2. DBeaver: горячие клавиши

DBeaver — клиент для работы с базой данных. Учебное подключение — база hh_school, схема hh_school.

Три сочетания клавиш, которые нужно запомнить

Комбинация	Что делает
Ctrl + Enter	Выполнить один запрос — тот, на котором стоит курсор
Alt + X (macOS: Cmd + X)	Выполнить весь скрипт целиком
Выделить фрагмент + Ctrl + Enter	Выполнить только выделенное

Основные элементы интерфейса

- **Menu bar** — меню основных команд
- **Toolbar** — часто используемые команды
- **Database Navigator** — дерево подключений: база → схема → таблицы, представления, функции
- **Editor** — рабочая область: SQL-редактор, редактор данных, ER-диаграммы
- **Results panel** — результаты запросов в виде вкладок-гридов
- **Status bar** — статус операции, число строк, время выполнения

3. Как устроены таблицы

Иерархия хранения данных

- **База данных (database)** — то, к чему вы подключаетесь. У нас — hh_school
- **Схема (schema)** — папка внутри базы для таблиц одной предметной области (public по умолчанию; hh_school — для учебных данных)
- **Таблица (table)** — данные, организованные в строки и колонки, например vacancies
- **Колонка (column)** — одно свойство объекта: имя + тип данных
- **Строка (row)** — один объект целиком. Одна строка в vacancies — одна вакансия

Типы данных

Категория	Тип	Пример
Числовые	INT, DECIMAL	Размер зарплаты, возраст сотрудника
Строковые	CHAR, VARCHAR	Имя, фамилия, описание
Дата и время	DATE, DATETIME / TIMESTAMP	2026-01-01 / 2026-01-01 11:11:11
Логические	BOOLEAN	Флаг «приглашён / удалён»

Первичный ключ (PK)

Первичный ключ — колонка (или набор колонок), которая уникально идентифицирует строку. Двух строк с одинаковым первичным ключом быть не может, и значение ключа не бывает пустым.

В таблице vacancies первичный ключ — vacancy_id. Название вакансии не подходит: «Аналитик данных» — обычное название, таких вакансий в базе полтора десятка, а vacancy_id уникален по построению.

> **СОВЕТ**
> Первичный ключ — это то, по чему считают «сколько у нас реально объектов».

3. Как устроены таблицы (продолжение)

Внешний ключ (FK) и связи

Внешний ключ — колонка, которая ссылается на первичный ключ другой таблицы. Это и есть механизм связывания данных.

- **1:1 «один к одному»** — не более одной строки на другой стороне. Пример: vacancy_details хранит объёмное описание отдельно от vacancies
- **1:N «один ко многим»** — самая частая связь. У одного работодателя много вакансий, у одной вакансии много откликов. Внешний ключ — на стороне «многих»
- **N:N «многие ко многим»** — вакансия требует несколько навыков, а навык встречается во многих вакансиях. Нужна промежуточная таблица: vacancy_id + skill_id

Схема учебной базы hh_school

Таблица	Строк	Одна строка — это
areas	5	Регион
employers	12	Компания-работодатель
vacancies	52	Вакансия
vacancy_details	45	Описание вакансии (есть не у всех)
responses	628	Отклик кандидата на вакансию
calls	251	Звонок между работодателем и кандидатом
messages	2862	Сообщение в переписке по отклику

Полная ER-схема: dbdiagram.io/d/hh-school-6aa7cbae36f99825648932d2

Воронка откликов (колонка responses.status): new (пришёл) → viewed (посмотрели) → invited / rejected (пригласили / отказали).

4. Первый SQL: SELECT, WHERE, ORDER BY, LIMIT

Анатомия запроса

```
SELECT   vacancy_id, vacancy_name, published_at  -- ЧТО вывести
FROM     vacancies                               -- ОТКУДА взять
WHERE    employment = 'Стажировка'               -- КАКИЕ строки оставить
ORDER BY published_at DESC                       -- КАК отсортировать
LIMIT    5;                                      -- СКОЛЬКО показать
```

Порядок, в котором запрос пишется, не совпадает с порядком, в котором он выполняется.

Порядок записи vs порядок исполнения

#	Пишем	Выполняет СУБД
1	SELECT	FROM
2	FROM	JOIN
3	JOIN	WHERE
4	WHERE	GROUP BY
5	GROUP BY	HAVING
6	HAVING	SELECT
7	ORDER BY	ORDER BY
8	LIMIT	LIMIT

WHERE: операторы

Оператор	Что означает	Пример
=, <>	Равно / не равно	experience = 'Нет опыта'
>, <, >=, <=	Больше / меньше	salary_from >= 80000
AND / OR	И одновременно / или	area_id = 1 OR area_id = 2
IN	Одно из нескольких значений	city IN ('Москва', 'Пермь')
BETWEEN	Попадает в диапазон	salary_from BETWEEN 80000 AND 150000
LIKE	Шаблон (% = любые символы)	vacancy_name LIKE 'Стажёр%'
IS NULL	Значение отсутствует	closed_at IS NULL

5. WHERE, ORDER BY, LIMIT

Подводные камни WHERE

ПОДВОДНЫЙ КАМЕНЬ

NULL — это отсутствие значения, а не 0 и не пустая строка. Нельзя написать WHERE closed_at = NULL — нужно WHERE closed_at IS NULL.

ПОДВОДНЫЙ КАМЕНЬ

AND выполняется раньше OR. WHERE city = 'Москва' OR city = 'Пермь' AND experience = 'Нет опыта' СУБД понимает как WHERE city = 'Москва' OR (city = 'Пермь' AND experience = 'Нет опыта'). Если нужна вакансия из двух городов и обязательно без опыта — скобки нужны явно: WHERE (city = 'Москва' OR city = 'Пермь') AND experience = 'Нет опыта'.

DISTINCT

Первый инструмент для исследования незнакомой колонки: какие значения вообще бывают и как записаны.

```
SELECT DISTINCT professional_role
FROM vacancies
ORDER BY professional_role;
```

ORDER BY и LIMIT

- ASC (по умолчанию) — по возрастанию, DESC — по убыванию. Можно сортировать по нескольким колонкам подряд или по номеру колонки: ORDER BY 2 DESC
- В PostgreSQL при ASC пустые значения — в конце, при DESC — в начале. Явно: ORDER BY salary_from DESC NULLS LAST
- LIMIT применяется **после** сортировки — связка ORDER BY … DESC LIMIT 10 — стандартный способ получить топ-10

```
SELECT vacancy_name, salary_from
FROM vacancies
ORDER BY salary_from DESC
LIMIT 10;
```

6. Агрегации и метрики: GROUP BY, HAVING

Метрика и измерение

- **Метрика** — что считаем: количество вакансий, средняя зарплата, конверсия
- **Измерение** — в каком разрезе считаем: по направлениям, городам, месяцам

Агрегатные функции

Функция	Что делает	Учитывает NULL?
count(*)	Считает строки	Да, все строки
count(колонка)	Считает непустые значения	Нет, пропускает
count(DISTINCT колонка)	Считает различные непустые значения	Нет
sum(колонка)	Сумма	Пропускает NULL
avg(колонка)	Среднее	Пропускает NULL
min / max	Минимум / максимум	Пропускают NULL

avg(salary_from) — это средняя зарплата среди вакансий, где она указана, а не среди всех вакансий. Формально функция отработала правильно, вопрос в том, на какой вопрос отвечает число.

GROUP BY

GROUP BY делит строки на группы по значению колонок, затем применяет агрегатную функцию внутри каждой группы.

```
SELECT professional_role,
       count(*) AS vacancies_cnt,
       count(DISTINCT employer_id) AS employers_cnt,
       round(avg(salary_from)) AS avg_salary_from
FROM vacancies
GROUP BY professional_role
ORDER BY vacancies_cnt DESC;
```

HAVING: фильтр после агрегации

WHERE фильтрует строки **до** группировки, HAVING фильтрует группы **после** неё.

```
SELECT professional_role, count(*) AS cnt
FROM vacancies
WHERE closed_at IS NULL          -- строки: только открытые вакансии
GROUP BY professional_role
HAVING count(*) >= 8             -- группы: только крупные направления
ORDER BY cnt DESC;
```

ПОДВОДНЫЙ КАМЕНЬ

Написать WHERE count(*) >= 8 нельзя: в момент выполнения WHERE группы ещё не построены.

7. JOIN: соединение таблиц

Условие соединения задаётся в ON и почти всегда выглядит одинаково: внешний ключ одной таблицы приравнивается к первичному ключу другой.

Виды JOIN

JOIN	Что делает	Когда используется
INNER JOIN	Только строки, для которых есть совпадение в обеих таблицах	Нужны только связанные записи
LEFT JOIN	Все строки левой таблицы + совпадения из правой	Нельзя терять строки основной таблицы
RIGHT JOIN	Все строки правой таблицы + совпадения из левой	То же самое, но основная таблица справа
FULL JOIN	Все строки из обеих таблиц	Нужно увидеть и совпадения, и записи без пары
CROSS JOIN	Каждая строка с каждой	Нужны все комбинации, например «дата × работодатель»

На практике аналитик чаще всего использует именно INNER JOIN и LEFT JOIN.

INNER JOIN — только совпадения

```
SELECT v.vacancy_name, e.employer_name
FROM vacancies v
INNER JOIN employers e
  ON v.employer_id = e.employer_id;
```

LEFT JOIN — сохраняем основную таблицу

```
SELECT e.employer_name,
       COUNT(r.response_id) AS responses
FROM employers e
LEFT JOIN vacancies v ON e.employer_id = v.employer_id
LEFT JOIN responses r ON v.vacancy_id = r.vacancy_id
GROUP BY e.employer_name;
```

8. Функции для аналитика

Работа с датами

- **date_trunc(единица, дата)** обрезает дату до начала периода — основной инструмент для динамики. Возвращает timestamp, поэтому результат обычно приводят к дате: ::date
- Две даты можно просто вычесть — результат: целое число дней
- Полезно помнить: current_date и арифметику с интервалами — published_at + INTERVAL '7 days'

```
SELECT date_trunc('month', published_at)::date AS month,
       count(*) AS vacancies
FROM vacancies
GROUP BY 1
ORDER BY 1;
```

CASE

CASE — это «если ... то ...» внутри запроса. Условия проверяются сверху вниз, срабатывает первое подошедшее.

```
SELECT CASE
        WHEN salary_from IS NULL   THEN 'не указана'
        WHEN salary_from <  60000  THEN 'до 60к'
        WHEN salary_from < 120000  THEN '60-120к'
        ELSE '120к+'
      END      AS salary_segment,
      count(*) AS cnt
FROM vacancies
GROUP BY 1
ORDER BY cnt DESC;
```

ПОДВОДНЫЙ КАМЕНЬ

Порядок условий важен: более узкое условие ставьте раньше. Проверку на NULL ставьте первой — иначе пустые значения молча попадут в ELSE.

NULL, COALESCE и NULLIF

- Любая арифметика с NULL даёт NULL, сравнение с NULL не истинно и не ложно, а агрегатные функции NULL просто игнорируют
- **coalesce(a, b, c, ...)** — первое непустое значение. Стандартный способ подставить значение по умолчанию
- **nullif(a, b)** — NULL, если a = b. Главное применение — защита от деления на ноль

```
round(100.0 * invited / nullif(responses, 0), 1)
```

FILTER: условный агрегат

FILTER (WHERE условие) применяется к агрегатной функции и считает только подходящие строки. В отличие от WHERE, FILTER действует локально — на один агрегат в конкретной колонке SELECT.

```
SELECT v.professional_role,
  count(r.response_id) AS responses,
  count(r.response_id) FILTER (WHERE r.status = 'invited') AS invited,
  round(100.0 * count(r.response_id) FILTER (WHERE r.status = 'invited')
        / nullif(count(r.response_id), 0), 1) AS conversion_pct
FROM vacancies v
JOIN responses r ON v.vacancy_id = r.vacancy_id
GROUP BY professional_role
ORDER BY conversion_pct DESC;
```

Где искать документацию

PostgreSQL — postgresql.org/docs

Trino — trino.io/docs/current/index.html

Spark — spark.apache.org/docs/latest/sql-ref.html

Автодополнение в клиенте: начните печатать имя функции. Запрос к ИИ или поисковику формулируйте по шаблону «название СУБД + что нужно сделать», например «postgres обрезать дату до месяца».

9. Шпаргалка

Каркас запроса

```
SELECT колонки
FROM таблица
[INNER|LEFT JOIN другая_таблица ON условие]
WHERE условие_на_строки
GROUP BY колонки
HAVING условие_на_группы
ORDER BY колонки [ASC|DESC]
LIMIT N;
```

Операторы WHERE

=, <>, >, <, >=, <=, AND, OR, IN (...), BETWEEN a AND b, LIKE '%шаблон%', IS NULL / IS NOT NULL

Агрегатные функции

count(*), count(колонка), count(DISTINCT колонка), sum(), avg(), min(), max()

JOIN

```
INNER JOIN b ON a.id = b.a_id
LEFT JOIN  b ON a.id = b.a_id
```

CASE

```
CASE WHEN условие1 THEN значение1
     WHEN условие2 THEN значение2
     ELSE значение_по_умолчанию
END
```

NULL-функции и FILTER

```
coalesce(a, b, ...)             -- первое непустое значение
nullif(a, b)                    -- NULL, если a = b
count(x) FILTER (WHERE условие) -- условный агрегат
```

Даты

```
date_trunc('month', dt)::date   -- обрезать до месяца
dt1 - dt2                       -- разница в днях
current_date, INTERVAL '7 days'
```